

SHADOWGRID — AUTONOMER FINALISIERUNGS-MASTERPROMPT FÜR CODEX

0. Rolle und Endziel

Du arbeitest als leitender Softwarearchitekt, Principal Engineer, Release Manager, QA-Leiter, Security Engineer, Accessibility Reviewer, Game-Balancing-Analyst, DevOps Engineer und Asset-Pipeline-Operator für SHADOWGRID.

Dein Auftrag ist, den bestehenden SHADOWGRID-Release-Candidate vom aktuellen Repositoryzustand bis zur technisch, visuell und operativ finalen Veröffentlichung zu führen.

Du baust das Projekt nicht neu. Du prüfst den realen Ist-Zustand, schließt jede belegte Lücke, testest jede Änderung und erzeugst am Ende einen reproduzierbaren finalen Release mit vollständiger Evidenz.

Das Ziel gilt erst als erreicht, wenn:

1. der vollständige lokale Stack reproduzierbar startet;
2. alle dokumentierten Spielsysteme funktionsfähig und automatisiert getestet sind;
3. alle Daten-, Ledger-, Eigentums-, Aktien- und Saisoninvarianten bestehen;
4. Web und Mobile technisch gebaut werden;
5. die vollständige Pflicht-Asset-Bibliothek erzeugt, validiert, optimiert und integriert ist;
6. Desktop- und Mobile-E2E-Tests grün sind;
7. WCAG 2.2 AA technisch und manuell soweit automatisierbar geprüft ist;
8. keine Critical- oder High-Sicherheitslücke offen bleibt;
9. Backup und Restore nachweislich funktionieren;
10. Release-, Operations-, Privacy-, Security-, Store- und Deployment-Dokumentation konsistent ist;
11. ein finaler Releasebericht mit Beweisen, offenen externen Operatoraktionen und exakten Befehlen existiert.

1. Verbindliche Quellen und Priorität

Lies vor jeder Änderung mindestens diese Dateien und behandle sie in folgender Reihenfolge als Quellen:

1. `ARCHITECTURE.md` — kanonische technische Wahrheit.
2. `AGENTS.md` — Repository- und Arbeitsregeln.
3. `PHASE_1_AUTH_ONBOARDING.md` bis `PHASE_12_RELEASE_HARDENING.md` — implementierte Fachverträge.
4. `SHADOWGRID_SPEC.md` — funktionale Spielvision.
5. `TRACEABILITY.md` — Zuordnung zwischen Anforderungen, Implementierung und Tests.
6. `RELEASE_NOTES_0.1.0-RC1.md` — verifizierter Ausgangsstand.
7. `RELEASE_CANDIDATE_FINDINGS.md` und gleichnamige aktuelle Varianten — Auditspur und Remediation.
8. `TESTING.md`, `ACCESSIBILITY.md`, `SECURITY_THREAT_MODEL.md`, `PRIVACY.md`.

9. `DEPLOYMENT.md`, `OPERATIONS_RUNBOOK.md`, `BACKUP_RESTORE.md`,
`MOBILE_RELEASE.md`.
10. `CODEX_ASSET_GENERATION_GOAL.md` — verbindliche visuelle Produktionspipeline.
11. `TRANSLATION_QUALITY.md` — tatsächlicher Übersetzungsstatus.

Bei Widersprüchen gilt:

- Die reale, getestete Implementierung schlägt eine ältere Roadmap-Annahme.
- `ARCHITECTURE.md` schlägt veraltete Stack-Vorgaben.
- Sicherheits- und Datenintegritätsregeln dürfen nicht abgeschwächt werden.
- Keine Dokumentation darf einen nicht belegten Abschluss behaupten.

Behalte FastAPI, ARQ, PostgreSQL, Redis, React, Expo und den modularen Monolithen bei. Führe keinen unnötigen Wechsel auf Flask, Celery, Socket.IO oder Microservices durch.

2. Autonomer Arbeitsmodus

Arbeite sequenziell, selbstständig und evidenzbasiert.

Du darfst keine routinemäßigen Rückfragen stellen. Triff sichere technische Entscheidungen selbst und dokumentiere sie.

Frage nicht nach Bestätigung für:

- normale Codeänderungen;
- Tests;
- lokale Migrationen;
- lokale Seeds;
- lokale Containerstarts;
- Dokumentationskorrekturen;
- nicht destruktive Refactorings;
- lokale Testdaten;
- automatisierte Qualitätsprüfungen.

Führe keine endgültige externe Aktion ohne explizites Repository-Flag aus. Verwende diese Flags:

```
FINALIZE_ALLOW_PRODUCTION_DEPLOY=false  
FINALIZE_ALLOW_GIT_PUSH=false  
FINALIZE_ALLOW_GIT_TAG=false  
FINALIZE_ALLOW_PAID_ASSET_GENERATION=false  
FINALIZE_ALLOW_STORE_SUBMISSION=false  
FINALIZE_ALLOW_EMAIL_PROVIDER_ACTIVATION=false
```

Nur ein exakt auf `true` gesetztes Flag erlaubt die jeweilige externe Aktion.

Fehlt ein Flag oder sind externe Zugangsdaten nicht verfügbar:

1. Blockiere nicht die übrige Arbeit.
2. Erstelle alle lokal möglichen Artefakte.
3. Führe Preview-, Dry-Run- oder lokale Prüfungen aus.

4. Schreibe einen präzisen Operator-Gate-Eintrag.
5. Dokumentiere den exakten letzten manuellen Schritt.
6. Setze mit allen anderen Phasen fort.

Speichere niemals Secrets im Repository, Frontend, Testreport, Screenshot, Log, Assetmanifest oder Promptarchiv. Maskiere gefundene Secrets, dokumentiere nur Dateipfad und Zeilennummer und ersetze sie durch Umgebungsvariablen.

Überschreibe keine unbekannten Benutzeränderungen. Prüfe zuerst `git status`, sichere den Ausgangszustand und arbeite auf einem eigenen Branch:

```
codex/finalize-shadowgrid
```

Erstelle nach jeder vollständig bestandenen Phase einen atomaren Commit. Pushen ist nur mit dem entsprechenden Flag erlaubt.

3. Fortschritts- und Wiederaufnahmesystem

Erstelle oder aktualisiere atomar:

```
.project/finalization-state.json  
.project/finalization-errors.json  
.project/finalization-decisions.md  
.project/finalization-evidence.md  
.project/finalization-summary.md
```

`finalization-state.json` muss mindestens enthalten:

```
{  
  "project": "SHADOWGRID",  
  "run_id": null,  
  "started_at": null,  
  "updated_at": null,  
  "current_phase": null,  
  "current_task": null,  
  "last_completed_phase": null,  
  "baseline_commit": null,  
  "working_branch": "codex/finalize-shadowgrid",  
  "release_version": null,  
  "phases": {},  
  "external_gates": [],  
  "critical_failures": [],  
  "final_status": "running"  
}
```

Jede Phase besitzt die Zustände:

```
pending
running
blocked_external
failed
completed
verified
```

Nach jeder Aufgabe:

1. Zustand speichern;
2. ausgeführte Befehle dokumentieren;
3. relevante Testresultate dokumentieren;
4. geänderte Dateien dokumentieren;
5. Fehler und Wiederholungen dokumentieren.

Bei einem Fehler:

1. Ursache analysieren;
2. gezielte Korrektur durchführen;
3. betroffene Prüfung erneut ausführen;
4. maximal drei automatische Reparaturversuche;
5. bei weiterem Fehler einen minimal reproduzierbaren Befund erstellen;
6. nur die betroffene Teilaufgabe als blockiert markieren;
7. unabhängige Aufgaben fortsetzen.

Kein Fehler darf still übersprungen werden.

PHASE 0 — Repositoryaufnahme und Schutz des Ausgangszustands

Ziel

Den realen Ist-Zustand sichern und eine belastbare Ausgangsbasis erzeugen.

Aufgaben

1. Lies alle verbindlichen Quelldokumente.
2. Erfasse:
3. Git-Branch;
4. Commit;
5. uncommittete Änderungen;
6. Node-, pnpm-, Python-, Docker-, PostgreSQL-, Redis- und Expo-Versionen;
7. Betriebssystem und Shell;
8. verfügbare Umgebungsdateien, ohne Secretwerte auszugeben.

9. Prüfe alle vorhandenen Scripts, Workflows, Dockerfiles und Releasebefehle.
10. Suche nach:
11. hart codierten Secrets;
12. ungetrackten produktionskritischen Dateien;
13. widersprüchlichen Dokumenten;
14. veralteten Stackreferenzen;
15. offenen `TODO`, `FIXME`, `HACK`, `XXX` ;
16. deaktivierten Tests;
17. `skip`, `xfail`, `only` und fokussierten Tests;
18. nicht reproduzierbaren Buildschritten.
19. Erzeuge einen lokalen, verifizierten Pre-Finalization-Backup.
20. Lege den Arbeitsbranch an.
21. Erzeuge:
22. `.project/current-state-audit.md` ;
23. `.project/source-of-truth-map.md` ;
24. `.project/open-gap-matrix.md` .

Gate

Phase 0 ist nur bestanden, wenn:

- der Ausgangszustand reproduzierbar dokumentiert ist;
- keine Benutzeränderung verloren ging;
- ein Backup existiert;
- alle Quellen priorisiert wurden;
- jede erkannte Lücke eine ID, Priorität, Beweis und Zielphase besitzt.

PHASE 1 — Release-Candidate-Baseline reproduzieren

Ziel

Den dokumentierten RC-Stand auf dem aktuellen Rechner und Commit erneut beweisen.

Aufgaben

Führe in sicherer Reihenfolge aus:

```
pnpm setup
pnpm data:verify
pnpm validate
```

Falls GNU Make vorhanden ist, verifiziere zusätzlich, dass:

```
make verify-release
```

denselben Workflow ausführt.

Starte außerdem den lokalen Stack entsprechend der vorhandenen Projektkonfiguration:

```
docker compose up --build -d postgres redis mailpit minio api worker web
prometheus
docker compose exec api python -m shadowgrid.predeploy
```

Prüfe:

```
GET /api/v1/health
GET /api/v1/ready
GET /healthz
GET /metrics nur aus erlaubter lokaler Umgebung
```

Kontrolliere Logs von API und Worker. Ein gesunder API-Prozess ohne laufenden ARQ-Worker ist ein Fehler.

Reparaturregeln

- Behebe jede Regression.
- Aktualisiere Lockfiles nur kontrolliert.
- Entferne keine Prüfung, um einen grünen Lauf zu erzwingen.
- Senke keine Coverage-, Security- oder Performancegrenze ohne dokumentierte technische Begründung.
- Dokumentiere bewusste Testskips einzeln.

Gate

Erforderlich:

- Migrationen grün;
- Seed idempotent;
- Dateninvarianten grün;
- Backend-, Web-, Mobile-, Load-, Security- und E2E-Gates grün;
- Web Production Build grün;
- Expo Export grün;
- keine neue Critical- oder High-Finding;

- vollständige Befehls- und Laufzeitevidenz.

PHASE 2 — Vollständige Gap-Analyse gegen alle Phasendokumente

Ziel

Beweisen, dass nicht nur Tests, sondern alle dokumentierten Anforderungen umgesetzt sind.

Aufgaben

Erzeuge eine maschinenlesbare und menschenlesbare Matrix:

```
Requirement ID
Quelldatei
Anforderung
Implementierungsdatei
API/Worker/UI-Bezug
Migration
Test
Status
Beweis
Restarbeit
```

Prüfe jede Anforderung aus Phase 1 bis Phase 12 sowie:

- Authentifizierung;
- Stadtwahl;
- Unternehmen;
- Wirtschaft;
- Spezialisten;
- KI;
- Börse;
- Kartelle;
- Informationen;
- strategisches PvP;
- Weltereignisse;
- Saisons;
- Verträge;
- Kredite;
- Anleihen;
- Immobilien;
- Realtime;
- Benachrichtigungen;
- Security;
- Datenschutz;

- Accessibility;
- Backup und Restore;
- Deployment;
- Mobile;
- Assetintegration.

Für jede echte Lücke:

1. implementieren;
2. migrieren;
3. API-Vertrag aktualisieren;
4. generierten Client aktualisieren;
5. Unit- und Integrationstest ergänzen;
6. Desktop- und Mobile-E2E ergänzen;
7. Dokumentation aktualisieren;
8. vollständigen betroffenen Gate-Lauf ausführen.

Gate

Kein Pflichtpunkt darf den Status `unknown`, `assumed` oder `untested` besitzen.

PHASE 3 — Perfekter lokaler Ein-Klick-Start

Ziel

Ein neuer Entwickler soll SHADOWGRID lokal ohne internes Wissen starten können.

Aufgaben

Stelle mindestens diese plattformgerechten Wege bereit:

```
Windows PowerShell
WSL2/Linux
Docker Compose
lokaler SQLite-Fallback, sofern bereits unterstützt
```

Erzeuge oder korrigiere:

```
scripts/setup-local.ps1
scripts/start-local.ps1
scripts/stop-local.ps1
scripts/reset-local.ps1
scripts/verify-local.ps1
scripts/setup-local.sh
scripts/start-local.sh
scripts/stop-local.sh
```



```
scripts/reset-local.sh  
scripts/verify-local.sh
```

Die Scripts müssen:

- Abhängigkeiten prüfen;
- fehlende lokale Env-Dateien sicher erzeugen;
- niemals echte Secrets ausgeben;
- Migrationen ausführen;
- Demo-Seed idempotent einspielen;
- API, Worker und Web starten;
- Health und Readiness prüfen;
- lokale URLs und Demo-Zugangsdaten ausgeben;
- verständliche Fehler liefern;
- wiederholbar sein.

Erzeuge eine `LOCAL_QUICKSTART.md` mit maximal zehn Hauptschritten und einen ausführlichen Troubleshooting-Abschnitt.

Gate

Ein sauberer lokaler Start muss aus einem frischen Checkout reproduzierbar sein. Danach müssen Login, Köln-Onboarding, Demo-KI und ein Wirtschaftstick funktionieren.

PHASE 4 — Vollständiger automatisierter Spieler-Lebenszyklus

Ziel

Den gesamten Spielweg von Kontoerstellung bis Saisonarchiv automatisiert beweisen.

Aufgaben

Erzeuge einen vollständigen API- und Playwright-E2E-Lauf mit mehreren Personas:

```
Unternehmer  
Investor  
Kartellführer  
Kartellmitglied  
Informationsstrategie  
Administrator  
lokaler KI-Spieler
```

Der Test muss mindestens abdecken:

1. Registrierung und Verifikation;
2. Login und Session-Rotation;
3. Köln und Bezirk auswählen;
4. Unternehmen gründen;
5. investieren;
6. Spezialisten einstellen und zuweisen;
7. Wirtschaftsticker und Gehalt;
8. KI-Entscheidung;
9. Börsengang;
10. Kauf- und Verkaufsaufträge;
11. Teilzahlung und Stornierung;
12. Dividende;
13. Kartellgründung;
14. Einladung und Beitritt;
15. Einzahlung und Vier-Augen-Ausgabe;
16. Kartellprojekt und Bezirkskontrolle;
17. Informationsoperation;
18. Berichtshandel;
19. abstrakte strategische Aktion;
20. Weltereignis-Vorschau und Aktivierung;
21. Ausschreibung, Angebot, Vergabe und Abrechnung;
22. Kreditangebot, Annahme und Rate;
23. Anleihe, Zeichnung, Kupon und Rückzahlung;
24. Immobilienkauf, Vermietung und Hauptquartier-Upgrade;
25. Realtime-Reconnect und Notification-Reconciliation;
26. Saisonphasen;
27. Saisonabschluss;
28. Hall of Fame und permanente Belohnung;
29. nächste Saison;
30. Erhalt der unveränderlichen Historie.

Prüfe nach jeder finanziellen Aktion die Ledger-Summe und nach jedem Aktiengeschäft die feste Gesamtaktienmenge.

Gate

Der komplette Lebenszyklus muss auf Desktop und mobilem Viewport ohne kritischen Axe-Befund bestehen.

PHASE 5 — UX-, Accessibility- und Responsive-Finalisierung

Ziel

Eine verständliche, hochwertige und barrierearme Oberfläche.

Aufgaben

Prüfe jede Hauptseite auf:

- klaren Primärzweck;
- sichtbare Lade-, Leer-, Fehler- und Erfolgszustände;
- genaue Kostenbestätigung;
- verständliche Fehlermeldungen mit Request-ID;
- vollständige Tastaturbedienung;
- korrekte Fokusreihenfolge;
- sichtbaren Fokus;
- 200-%-Zoom;
- Reduced Motion;
- hohe Kontraste;
- Textalternative für Karten und Netzwerke;
- semantische Tabellen;
- mobile Touch-Ziele;
- RTL;
- `en-XA` und `ar-XB`;
- deutsche und englische Parität.

Führe automatisierte Axe-Prüfungen aus und ergänze eine manuelle Checkliste für:

- Screenreader-Lesereihenfolge;
- Dialogfokus;
- mobile Textskalierung;
- iOS- und Android-Touchbedienung;
- Verständlichkeit wirtschaftlicher Begriffe.

Behebe alle Critical- und Serious-Axe-Funde. Dokumentiere begründete Ausnahmen.

Gate

`ACCESSIBILITY_FINAL_REPORT.md` muss den Status jeder Seite und jeder manuellen Prüfung enthalten.

PHASE 6 — Vollständige autonome Asset-Pipeline

Ziel

Die in `CODEX_ASSET_GENERATION_GOAL.md` definierte visuelle Bibliothek vollständig erzeugen und integrieren.

Ausführung

1. Analysiere vorhandene Assets.

2. Erzeuge `assets/asset-manifest.json`.
3. Erzeuge den Style Lock.
4. Generiere zuerst ausschließlich den Style-Proof.
5. Prüfe den Style-Proof intern gegen:
6. Realismus;
7. deutsche Architektur;
8. Materialqualität;
9. Licht;
10. Cyber-Noir-Anteil;
11. zurückhaltende Goldakzente;
12. UI-Freiraum;
13. Mobile-Eignung;
14. Originalität;
15. Sicherheitskonformität.
16. Friere den Style Lock erst nach bestandener Prüfung ein.
17. Verarbeite danach exakt die Manifestreihenfolge.
18. Speichere Prompt, Seed, Provider, Version, Hash, Lizenzstatus, Fokuspunkt, Safe Area und Qualitätswerte.
19. Erzeuge responsive AVIF-, WebP-, PNG- und SVG-Varianten nach Vorgabe.
20. Integriere jedes freigegebene Asset sofort in die reale Oberfläche.
21. Teste Pfade, Formate, Ladeverhalten, Crops und Performance.
22. Erzeuge nach jedem Batch Kontaktbögen.
23. Regeneriere Ausreißer.

Providerregeln

- Nutze kostenpflichtige Provider nur bei `FINALIZE_ALLOW_PAID_ASSET_GENERATION=true`.
- Halte Tages- und Gesamtbudget ein.
- Fehlt die Freigabe oder ein Provider, erzeuge hochwertige prozedurale Premium-Fallbacks.
- Kennzeichne Fallbacks korrekt.
- Kein Platzhalter und keine einfache Farbfläche darf als fertig gelten.
- Kein reales Firmen-, Behörden- oder Organisationslogo.
- Keine reale kriminelle Organisation.
- Keine Gewalt- oder Kriminalitätsanleitung.
- Keine erfundene Lizenzangabe.

Pflichtberichte

```
assets/reports/ASSET_MANIFEST_REPORT.md
assets/reports/STYLE_CONSISTENCY_REPORT.md
assets/reports/IMAGE_QUALITY_REPORT.md
assets/reports/IMAGE_SAFETY_REPORT.md
assets/reports/ASSET_LICENSE_REPORT.md
assets/reports/ASSET_PERFORMANCE_REPORT.md
assets/reports/MOBILE_CROP_REPORT.md
assets/reports/ASSET_INTEGRATION_REPORT.md
```

```
assets/reports/ASSET_GENERATION_COST_REPORT.md
assets/reports/FINAL_ASSET_COVERAGE.md
```

Gate

Kein verpflichtendes Asset darf `missing`, `rejected` oder undokumentiert sein.

`review_required` ist nur zulässig, wenn ein sicherer Produktionsfallback aktiv ist und der externe Reviewpunkt exakt dokumentiert wurde.

PHASE 7 — Echte Store-, Marketing- und Community-Materialien

Ziel

Marketingmaterial ausschließlich aus dem realen Spielzustand erzeugen.

Aufgaben

Starte den echten lokalen oder freigegebenen Preview-Build und erzeuge:

- Google-Play-Screenshots;
- iPhone-Screenshots;
- iPad-Screenshots;
- Feature Graphic;
- Open-Graph-Grafik;
- Community-Banner;
- Discord-Banner;
- Saisonstart-, Saisonfinale-, Alpha-, Beta- und Release-Banner.

Verwende keine erfundenen UI-Screenshots. Jeder Screenshot muss aus einer funktionierenden Spieloberfläche stammen.

Prüfe:

- korrekte Sprache;
- keine Demo-Secrets;
- keine realen E-Mail-Adressen;
- keine privaten IDs;
- keine Debuganzeigen;
- keine falschen Leistungsversprechen;
- sichere Crops;
- Store-Abmessungen;
- visuelle Konsistenz.

Gate

Erzeuge `STORE_ASSET_READINESS.md` mit Datei, Abmessung, Quelle, Sprache, Plattform und Freigabestatus.

PHASE 8 — Balancing, Simulation und Spielspaß-Risiken

Ziel

Technische Korrektheit in belastbares Spielbalancing überführen.

Aufgaben

Erzeuge deterministische Langzeitsimulationen mit mehreren Strategien und mindestens:

100 simulierte Spieler
500 Unternehmen
mehrere konkurrierende Kartelle
mehrere vollständige Saisons

Analysiere:

- Vermögensverteilung;
- Marktanteilskonzentration;
- Zeit bis erste Firma;
- Zeit bis erster Gewinn;
- Zeit bis erster Spezialist;
- Zeit bis Börsengang;
- Aktienliquidität;
- Kartelldominanz;
- Comeback-Chancen;
- Insolvenzquote;
- Vertragsausfälle;
- Kredit- und Anleiheausfälle;
- Immobilienkonzentration;
- Nutzen jeder Investition;
- neue Spieler gegenüber frühen Spielern;
- Stärke jeder KI-Strategie;
- dominante oder wertlose Strategie;
- Saisonlänge;
- Anzahl sinnvoller Entscheidungen.

Balancingänderungen müssen:

- datengetrieben;

- versioniert;
- konfigurierbar;
- reproduzierbar;
- durch Regressionstests abgesichert sein.

Ändere keine historischen Snapshots.

Gate

Erzeuge:

```
BALANCE_SIMULATION_REPORT.md
BALANCE_CHANGELOG.md
SEASON_0_BALANCE_CONFIG.md
```

Keine einzelne Strategie darf ohne erheblichen Nachteil dauerhaft dominant sein. Kritische Wirtschaftsexploits müssen geschlossen sein.

PHASE 9 — Security-, Privacy- und Compliance-Finalisierung

Ziel

Öffentliche Veröffentlichung technisch und dokumentarisch vorbereiten.

Aufgaben

Führe erneut aus:

- Secret Scan;
- Bandit;
- pip-audit;
- pnpm audit;
- Dependency Review;
- Auth- und Session-Tests;
- IDOR-Tests;
- Rate-Limit-Tests;
- Input-Limit-Tests;
- CORS-Tests;
- Backup- und Restore-Tests;
- Ledger- und Aktieninvarianten.

Prüfe die dokumentierte React-Router-Ausnahme. Entferne die Ausnahme, falls inzwischen RSC- oder Framework-Modus verwendet wird.

Datenschutz:

- Exportfunktion testen;
- Lösch- und Pseudonymisierungsworkflow testen;
- Logs auf verbotene Inhalte prüfen;
- Analytics weiterhin standardmäßig deaktivieren;
- Processor- und Retention-Platzhalter eindeutig markieren;
- Supportkontakt als externes Pflichtfeld markieren;
- keine juristische Freigabe vortäuschen.

Erzeuge:

```
SECURITY_FINAL_REPORT.md  
PRIVACY_LAUNCH_CHECKLIST.md  
DATA_RETENTION_MATRIX.md  
PROCESSOR_REGISTER_TEMPLATE.md  
INCIDENT_RESPONSE_CHECKLIST.md
```

Gate

- 0 offene Critical;
- 0 offene High;
- jede Medium-Abweichung behoben oder mit Owner, Risiko und Frist akzeptiert;
- keine Secret-Leaks;
- Datenschutzlücken als externe Legal- oder Operator-Gates klar markiert.

PHASE 10 — Operations, Backup, Monitoring und Deployment-Härtung

Ziel

Sicherer Betrieb und Wiederherstellung.

Aufgaben

1. Prüfe Readiness, Liveness, Request-IDs und strukturierte Logs.
2. Prüfe, dass API und Worker gemeinsam überwacht werden.
3. Beschränke `/metrics` in Reverse Proxy, Railway-Netz oder Firewall-Konfiguration.
4. Prüfe Alertdefinitionen für:
5. Readiness;
6. 5xx;
7. p95-Latenz;
8. Workerfehler;

9. Redis;
10. PostgreSQL;
11. Ledger-Reconciliation.
12. Führe lokalen Backup- und Restore-Drill aus.
13. Prüfe SHA-256 beziehungsweise PostgreSQL-Dump-Verifikation.
14. Führe nach Restore `pnpm data:verify` aus.
15. Erzeuge einen Season-Open- und Season-Close-Runbook-Test.
16. Prüfe die Rollbackstrategie.
17. Erzeuge Staging-Smoke-Test und Produktions-Smoke-Test als automatisierbare Scripts.

Produktionsregel

Nur bei:

`FINALIZE_ALLOW_PRODUCTION_DEPLOY=true`

darf ein Deployment ausgelöst werden.

Vor Deployment zwingend:

1. verifiziertes Backup;
2. vollständiges Release-Gate;
3. rückwärtskompatible Migration;
4. Worker- und API-Readiness;
5. Auth-Smoke;
6. Produktionswelt-Smoke;
7. Logout-Smoke;
8. `pnpm data:verify`.

Ohne Flag:

- Deploymentplan;
- Dry Run;
- exakte Befehle;
- Checkliste;
- Rollbackplan;
- Status `blocked_external`.

Gate

`OPERATIONS_FINAL_READINESS.md`

 enthält jeden Nachweis und jeden noch externen Schritt.

PHASE 11 — Mobile-Signierung und Store-Vorbereitung

Ziel

Android und iOS vollständig vorbereiten.

Aufgaben

1. Platzhalter-EAS-Projekt-ID und Beispieldomains erkennen.
2. Produktions-API-URL validieren.
3. Bundle IDs prüfen.
4. Preview-Build erzeugen, soweit ohne externe Credentials möglich.
5. Expo Export und Mobile Tests ausführen.
6. Store Copy, Datenschutzangaben und Data-Safety-Entwurf prüfen.
7. Gerätecheckliste erzeugen:
8. Login;
9. Session Restore;
10. Deep Links;
11. Offline;
12. Dark Mode;
13. Text Scaling;
14. Screenreader;
15. Touch Targets.
16. Signed AAB oder IPA nur bei verfügbaren Credentials und `FINALIZE_ALLOW_STORE_SUBMISSION=true`.
17. Zuerst interne Tracks, danach Staged Rollout.

Ohne Storefreigabe:

- alle Buildkonfigurationen fertigstellen;
- exakte EAS-Befehle dokumentieren;
- fehlende Credential- und Accountschritte dokumentieren;
- keine Schlüssel oder Zertifikate im Repository erzeugen.

Gate

`MOBILE_RELEASE_READINESS.md` trennt klar:

```
repository_complete
preview_build_complete
signed_build_external
store_submission_external
```

PHASE 12 — Transaktionale E-Mail und öffentliche Kontoabläufe

Ziel

Registrierung, Verifikation und Passwort-Reset unter realistischen Bedingungen beweisen.

Aufgaben

- Mailpit lokal vollständig testen;
- SMTP-Konfiguration validieren, ohne Secrets auszugeben;
- Links auf korrekte öffentliche URL prüfen;
- Verifikationsablauf testen;
- Passwort-Reset testen;
- Ablauf, Replay und invalidierte Tokens testen;
- E-Mail-Inhalte auf Deutsch und Englisch prüfen;
- Rate Limits und Abuse-Schutz prüfen.

Produktiven Provider nur aktivieren bei:

```
FINALIZE_ALLOW_EMAIL_PROVIDER_ACTIVATION=true
```

Sonst `SMTP_PROVIDER_OPERATOR_GATE.md` mit exakten Variablen und Prüfungen erzeugen.

PHASE 13 — Finaler vollständiger Release-Lauf

Ziel

Einen unverfälschten finalen Build aus sauberem Zustand beweisen.

Ablauf

1. Arbeitsbaum bereinigen, ohne Benutzerdaten zu löschen.
2. Abhängigkeiten aus Lockfiles installieren.
3. Frische Datenbank erzeugen.
4. Vollständige Migration ausführen.
5. Seed zweimal ausführen.
6. Dateninvarianten prüfen.
7. Vollständiges `pnpm validate` ausführen.
8. Assetvalidierung ausführen.
9. Assetintegrationstest ausführen.
10. Balancingsimulation ausführen.
11. Backup erstellen.
12. Restore durchführen.
13. Dateninvarianten erneut prüfen.

14. Lokalen Full-Life-Cycle-E2E ausführen.
15. Web Production Build erzeugen.
16. Expo Export erzeugen.
17. Security Scan ausführen.
18. Accessibility Gate ausführen.
19. Dokumentations-Linkcheck ausführen.
20. Lizenz- und Secret-Scan ausführen.

Erzeuge eine unveränderliche Evidenzzusammenfassung mit:

- Commit;
- Branch;
- Zeit;
- Toolversionen;
- Befehlen;
- Exitcodes;
- Testanzahl;
- Coverage;
- Buildgrößen;
- Load-Ergebnissen;
- Assetabdeckung;
- Securitystatus;
- Backuphash;
- offenen externen Gates.

Gate

Kein Pflichtgate darf rot sein.

PHASE 14 — Releaseversion, Changelog und Promotion

Ziel

Den finalen Release reproduzierbar paketieren.

Aufgaben

Bestimme die nächste Version nach bestehender Projektkonvention, beispielsweise:

0.1.0
oder
0.1.0-rc.2

Erzeuge:

```
RELEASE_NOTES_<VERSION>.md
CHANGELOG.md
FINAL_RELEASE_MANIFEST.json
FINAL_RELEASE_EVIDENCE.md
FINAL_OPERATOR_CHECKLIST.md
FINAL_EXTERNAL_GATES.md
```

`FINAL_RELEASE_MANIFEST.json` enthält mindestens:

```
{
  "version": "",
  "commit": "",
  "builds": {
    "web": "",
    "mobile_preview": "",
    "android_signed": "external_or_complete",
    "ios_signed": "external_or_complete"
  },
  "tests": {},
  "security": {},
  "assets": {},
  "backup": {},
  "deployment": {},
  "external_gates": []
}
```

Tag nur bei:

```
FINALIZE_ALLOW_GIT_TAG=true
```

Push nur bei:

```
FINALIZE_ALLOW_GIT_PUSH=true
```

Produktion nur bei:

```
FINALIZE_ALLOW_PRODUCTION_DEPLOY=true
```

Nach freigegebenem Produktionsdeployment:

- Readiness prüfen;
- API- und Worker-Logs prüfen;
- Auth testen;
- Welt lesen;
- eine sichere, nicht destruktive Spieleraktion testen;

- Logout testen;
- `pnpm data:verify` ausführen;
- Produktions-Smokereport erzeugen.

PHASE 15 — Abschlussbericht und eindeutige Endzustände

Erzeuge

```
FINAL_SHADOWGRID_COMPLETION_REPORT.md
FINAL_SHADOWGRID_TECHNICAL_REPORT.md
FINAL_SHADOWGRID_GAMEPLAY_REPORT.md
FINAL_SHADOWGRID_ASSET_REPORT.md
FINAL_SHADOWGRID_SECURITY_REPORT.md
FINAL_SHADOWGRID_ACCESSIBILITY_REPORT.md
FINAL_SHADOWGRID_OPERATIONS_REPORT.md
FINAL_SHADOWGRID_EXTERNAL_ACTIONS.md
```

Erlaubte Endzustände

COMPLETE_LOCAL_AND_REPOSITORY

Alle Repository-, lokalen, Test-, Build-, Asset-, Security-, Accessibility- und Dokumentationsarbeiten sind abgeschlossen. Nur externe Konten oder Freigaben fehlen.

COMPLETE_RELEASED

Zusätzlich sind freigegebenes Deployment, produktives SMTP und gegebenenfalls signierte Store-Builds abgeschlossen.

BLOCKED_CRITICAL

Nur zulässig, wenn ein nicht automatisch lösbarer Critical-Befund existiert. Dokumentiere:

- exakte Ursache;
- Reproduktion;
- Sicherheitsauswirkung;
- betroffene Dateien;
- bereits versuchte Reparaturen;
- kleinstmögliche erforderliche externe Entscheidung.

Abschlussnachricht

Am Ende antworte nicht nur mit „fertig“.

Liefere:

1. finalen Status;
 2. Releaseversion und Commit;
 3. was implementiert oder repariert wurde;
 4. welche Tests bestanden;
 5. Assetabdeckung;
 6. Securitystatus;
 7. Accessibilitystatus;
 8. Backup- und Restore-Nachweis;
 9. lokale Startbefehle;
 10. Produktionsstatus;
 11. Storestatus;
 12. verbleibende externe Schritte mit exakten Befehlen.
-

Unverhandelbare Qualitätsregeln

- Keine Behauptung ohne Beweis.
- Keine still deaktivierten Tests.
- Keine Entfernung von Sicherheitsprüfungen für einen grünen Build.
- Keine Geldberechnung mit Floating Point.
- Keine direkte Manipulation von Konten, Eigentum oder Aktien außerhalb der Domainservices.
- Keine Clientautorität über Spielzustand.
- Keine ungeschützten Secrets.
- Keine erfundenen Store-Screenshots.
- Keine realen Marken oder Behördenlogos in Assets.
- Keine realen kriminellen Organisationen oder Verfahrensanleitungen.
- Keine destruktive Produktionsaktion ohne Flag und Backup.
- Keine Datenbank-Handkorrektur von Ledger-, Aktien- oder Eigentumszeilen.
- Keine Überschreibung historischer Snapshots.
- Keine Übersetzung als fertig markieren, wenn nur englischer Fallback existiert.
- Keine Accessibility-Freigabe ausschließlich auf Basis automatisierter Axe-Tests.
- Kein Abschluss, solange Pflichtberichte fehlen.

Beginne jetzt mit Phase 0. Arbeite ohne routinemäßige Rückfragen bis zum höchstmöglichen autonomen Endzustand weiter.